

Session 3

Terminal Commands & Environment Setup



Talk to your computer in words, and save every working version.

Part	Topic	Time
01	What the terminal is	4 min
02	Moving around: pwd, ls, cd	7 min
03	Making and changing things: mkdir, cp, mv, rm	7 min
04	A tidy project folder	3 min
05	Git and GitHub: save points for your code	9 min

Ten words you will hear today.

Word	Plain meaning
Terminal	An app where you type commands instead of clicking.
Command	One instruction, like ls. Press Return to run it.
Folder (directory)	A box that holds files and other folders. Same thing, two names.
Path	The directions to a file or folder, like /Users/alex/incipe-projects.
Home folder	Your own folder on the Mac. Written ~ for short.
Current folder	The folder the terminal is "standing in" right now.
Git	A tool that saves versions of your project so you can go back.
Repository (repo)	A project folder that Git is looking after.
Commit	One saved version, with a short message saying what changed.
GitHub	A website that keeps a copy of your repository online.

Texting your computer instead of tapping it.

Type exactly what you want, press Return, and it answers. Open it: press ⌘ Space, type Terminal, press Return.

```
alex@MacBook ~ %
```

Part	Meaning
alex	Who is signed in
MacBook	The computer's name
~	The current folder — ~ means your home folder
%	"Ready. Type a command."

Folders are rooms. A path is the directions.

/Users/alex/incipe-projects means: start at the front door, go into Users, then alex, then incipe-projects.

Command	Say it as	What it does
pwd	"print working directory"	Shows the full path of the room you are in
ls	"list"	Shows what is in this room
cd name	"change directory"	Walks into the room called name
cd ..	"go up"	Walks back out to the room around this one
cd ~ or cd	"go home"	Jumps straight back to your home folder

Alex makes a home for their projects.

```
% pwd
/Users/alex
% mkdir incipe-projects
% cd incipe-projects
% mkdir climate-reader
% ls
climate-reader
% cd climate-reader
% pwd
/Users/alex/incipe-projects/climate-reader
% cd ..
```

WHAT HAPPENED

A new window starts in your home folder.

mkdir makes a folder and prints nothing when it works.

cd walks in; pwd checks where you are.

cd .. walks back out to /Users/alex/incipe-projects.

Make, copy, move, remove.

Command	What it does	Example
mkdir name	Makes a new folder	mkdir sketches
touch name	Makes a new, empty file	touch notes.txt
cp from to	Copies a file — the original stays	cp notes.txt backup.txt
mv from to	Moves or renames — the original is gone	mv backup.txt old-notes.txt
rm name	Removes (deletes) a file	rm old-notes.txt

Copy, rename, delete — and check with ls.

```
% touch notes.txt
% cp notes.txt backup.txt
% ls
backup.txt  notes.txt
% mv backup.txt old-notes.txt
% ls
notes.txt  old-notes.txt
% rm old-notes.txt
% ls
notes.txt
```

WHAT HAPPENED

cp made a second file; the original stayed.

mv renamed backup.txt. No copy was left behind.

rm deleted old-notes.txt. Only notes.txt is left.

ls lists in alphabetical order.

rm does not use the Bin.

A file you rm is gone for good — there is no undo. Check the name twice before you press Return.

To delete a folder you need `rm -r name`: the folder and everything in it. Be even more careful with that one.

The terminal tells you what went wrong in one line.

You typed	It said	Why
cd blink	cd: no such file or directory: blink	There is no folder called blink here. Check with ls.
mkdir climate-reader	mkdir: climate-reader: File exists	That folder is already there.
rm climate-reader	rm: climate-reader: is a directory	It is a folder, so plain rm refuses. Nothing was deleted.

One folder for everything, one folder per project.

```
incipe-projects/  
  climate-reader/  
    climate-reader.ino  
    notes.md  
    prompts.md
```

Rule	Why
Lowercase words joined with -	A name with spaces needs quotes in the terminal: <code>cd "climate reader"</code>
One project per folder	Git looks after one folder at a time
Keep a prompts.md beside the code	The prompts that worked are worth reusing (Lesson 1)

Git gives your code save points.

Like a video game: save before the hard level, and if it goes wrong, load the save and try again. Each save point is a commit, with a label saying what changed.

Tell Git who you are.

```
% git config --global user.name "Alex Chan"  
% git config --global user.email "alex@example.com"
```

Line	What it does
1	Sets the name shown on your commits. Use your own.
2	Sets the email shown on your commits. Use the one you will use for GitHub.

Init once. Then add and commit.

```
% git init
Initialized empty Git repository in /Users/alex/incipe-projects/climate-reader/.git/
% git status
On branch main
No commits yet
Untracked files:
  climate-reader.ino
  notes.txt
% git add climate-reader.ino
% git commit -m "First version of the climate reader"
[main (root-commit) 68f8b60] First version of the climate reader
 1 file changed, 2 insertions(+)
% git log --oneline
68f8b60 First version of the climate reader
```

Change, add, commit — small and often.

Command	What it does
git init	Starts Git in this folder. Once per project.
git status	Shows what changed and what is ready to save. Run it often.
git add file	Puts a file into the next save point.
git commit -m "message"	Makes the save point. The message says what changed.
git log --oneline	Lists save points, one per line, newest first.

Every time after the first: change your code → git add the files → git commit -m "what changed".

Git saves on your computer. GitHub keeps a copy online.

Step	Do this
1	Make a GitHub account at github.com . Accounts are for ages 13 and up — check with your teacher first.
2	On GitHub, create a new, empty repository called <code>climate-reader</code> .
3	Connect your folder to it, once: <code>git remote add origin https://github.com/your-name/climate-reader.git</code>
4	Send your save points up: <code>git push -u origin main</code> (use <code>master</code> if that is your branch).

The first push asks you to sign in to GitHub — your teacher will show you how your school does it. After that, `git push` on its own sends new commits.

Never run a command you cannot explain.

1

AI is good at explaining commands. Ask before you run one:
"Explain each part of this command in one line, for a beginner. Do not suggest anything else."

2

AI can also suggest commands that delete things. If it suggests `rm -r` or `rm -rf`, stop and ask your teacher.

Build your project home, then save it.

1. Open Terminal. Run `pwd` and write down your home folder's path.
2. Make `incipe-projects`, and `climate-reader` inside it, with `mkdir` and `cd`.
3. Inside `climate-reader`, make `notes.md` and `prompts.md` with `touch`. Check with `ls`.
4. Copy `notes.md` to `notes-backup.md`, rename the copy to `old-notes.md`, then delete it. Run `ls` after each step.
5. Set your Git name and email, then `git init`, `git add notes.md prompts.md`, and make your first commit. Check it with `git log --oneline`.

Hand in one screenshot of your Terminal.

#	Show	It proves
1	pwd inside climate-reader	You can find where you are
2	ls with notes.md and prompts.md	You can make files and folders
3	git log --oneline with at least two commits	You can save versions
4	Your prompts.md holding one prompt from Lesson 1 or Session 2	You keep what works

You are in `/Users/alex/incipe-projects`. What prints?

READ THE COMMANDS

```
cd climate-reader  
pwd
```

ANSWER

`/Users/alex/incipe-projects/
climate-reader`

WHY

`cd` walked one folder deeper, and `pwd` prints the full path.

You are in the climate-reader folder. What prints?

READ THE COMMANDS

```
pwd
/Users/alex/incipe-projects/climate-reader
cd ..
pwd
```

ANSWER

/Users/alex/incipe-projects

WHY

.. is the folder around this one — one level up.

The folder holds only notes.txt. What does ls print?

READ THE COMMANDS

```
cp notes.txt backup.txt  
mv backup.txt old-notes.txt  
ls
```

ANSWER

notes.txt old-notes.txt

WHY

cp made backup.txt, then mv renamed it. ls lists in alphabetical order.

climate-reader is a folder. What happens?

READ THE COMMAND

```
rm climate-reader
```

ANSWER

Nothing is deleted

WHY

It prints `rm: climate-reader: is a directory`. Plain `rm` only removes files; a folder needs `rm -r`.

You changed the sketch and skipped git add. Is a save point made?

READ THE COMMAND

```
git commit -m "Add loop"
```

ANSWER

No

WHY

Git prints no changes added to commit (use "git add" and/or "git commit -a"). Nothing goes into a commit until you git add it.

Three things to take away.

1

`pwd` says where you are, `ls` what is here, `cd` moves you. `..` is one level up and `~` is home.

2

`mkdir`, `cp`, `mv` and `rm` make, copy, move and delete — and `rm` has no undo.

3

Git saves versions: `git init` once, then `add` and `commit` every time. GitHub keeps a copy online.

Next — Session 4: Command line for embedded development. Using the terminal to compile and upload firmware.